

AI-driven Smart User Interface Generation from UML Models for Model-Driven Engineering

Ruike Yuan

First supervisor: Guus Ramakers

Second supervisor: Jan van Rijn

Third supervisor: Lisa Tweedie

BACHELOR THESIS

Leiden Institute of Advanced Computer Science (LIACS)

www.liacs.leidenuniv.nl

17/02/2026

Contents

1	Introduction	1
1.1	Problem Statement	1
1.1.1	Research Problem	1
1.1.2	Importance of the Problem	1
1.1.3	Academic Contribution	1
1.2	Research Goals and Hypotheses	2
1.2.1	Research Objective	2
1.2.2	Hypotheses	2
2	Background and Related Work	2
2.1	Model-Driven Engineering (MDE)	2
2.2	UML Models in Software Development	2
2.2.1	Class Diagrams	3
2.2.2	Use Case Diagrams	3
2.2.3	Activity Diagrams	3
2.3	UI Generation from Models	3
2.3.1	Rule-Based Approaches	3
2.3.2	Template-Based Approaches	3
2.4	Object-Oriented User Interface (OOUI) Principles	3
2.5	Large Language Models in Software Engineering	4
2.5.1	LLM Capabilities and Limitations	4
2.5.2	LLM for Code/UI Generation	4
2.6	Human-in-the-Loop (HITL) Approaches	4
2.7	The AI4MDE Framework	4
2.8	Summary and Research Gap	4
3	Research Approach and Methods	5
3.1	Methodology	5
3.2	Input Data	6
3.3	LLM-based UI Generation	7
3.4	Human-in-the-Loop Iteration	7
3.5	Verification and Evaluation	7
3.6	Summary	7
4	Evaluation and Results	8
4.1	Evaluation Methodology	8
4.1.1	Participants	8
4.1.2	Test Scenarios	8
4.1.3	Evaluation Dimensions	8
4.2	OOUI Alignment Evaluation	9
4.2.1	Evaluation Criteria	9
4.2.2	Assessment Method	9
4.2.3	Expected Results	9

4.3	Usability Evaluation	9
4.3.1	Task-Based Testing	9
4.3.2	Metrics	10
4.3.3	Comparison Baseline	10
4.4	Variant Diversity Analysis	10
4.4.1	Quantitative Metrics	10
4.4.2	Qualitative Assessment	10
4.5	Human-in-the-Loop Effectiveness	10
4.5.1	Iteration Analysis	10
4.5.2	Convergence Analysis	11
4.6	Results Summary	11
5	Discussion	11
5.1	Interpretation of Results	11
5.1.1	H1: OOUI Principle Effectiveness	11
5.1.2	H2: Enterprise Application Suitability	11
5.1.3	H3: Human-in-the-Loop Refinement	12
5.2	Threats to Validity	12
5.2.1	Internal Validity	12
5.2.2	External Validity	12
5.2.3	Construct Validity	12
5.3	Limitations	12
5.3.1	LLM Limitations	12
5.3.2	Architectural Limitations	13
5.4	Comparison with Related Work	13
6	Conclusion	14
6.1	Summary of Contributions	14
6.2	Answers to Research Hypotheses	14
6.2.1	H1: OOUI Principle Reflection	14
6.2.2	H2: Enterprise Application Suitability	15
6.2.3	H3: Human-in-the-Loop Effectiveness	15
6.3	Future Work	15
6.4	Final Remarks	16

1 Introduction

Executive Summary

Generating user interfaces from UML models is a well-established topic in Model-Driven Engineering (MDE). Existing research in generating user interfaces from UML models, such as class, use case and activity models, typically uses a rule-based approach to generate web user interfaces .

Model-Driven Engineering enables the systematic generation of software artifacts from high-level models such as UML. AI4MDE is a model-driven engineering framework that supports the automated generation of software artifacts from UML models. Within the AI4MDE framework, there is an existing UI code generator that can convert UML class models, use case models and activity models into web user interfaces. However, the current generator is rule-based, which means they have limited expressiveness and are difficult to adapt in complicated enterprise workflows and multi-user interaction scenarios.

This thesis offers an AI-driven improvement by integrating Large Language Models (LLMs) into the current generation pipeline to generate diverse, context-driven UI metadata guided by Object-Oriented principles.

The research focuses on improving the quality, usability and adaptability of automatically generated user interfaces while maintaining compatibility with the current AI4MDE generator architecture.

1.1 Problem Statement

1.1.1 Research Problem

Automated Interface generation in model-driven engineering aims to improve development efficiency by transforming high-level UML models.

However, the current AI4MDE interface is generated relying on fixed rules that map from UML elements [vD24, Wil24] to a preliminary user interface metadata, which is not able to understand the context, user roles, workflows and design intent. This leads to monotonous interfaces that are difficult to adapt to enterprise-level usability and Object-Oriented User Interface (OOUI) design requirements [Ver25].

1.1.2 Importance of the Problem

Software systems in enterprises often involve multiple actors and complicated procedures [Wil24]. It is time-consuming and easy to make mistakes when manually designing interactive interfaces [vA22], while rule-based generators have difficulties in handling details [Ver25]. With the help of AI to generate interfaces, consistency can be improved, development costs can be reduced, and high-quality user interfaces can be directly produced from models [Zei23].

1.1.3 Academic Contribution

- Investigate how LLM-based reasoning in the process of generating user interfaces within a model-driven engineering (MDE) generation pipeline can innovate and extend the existing methodology.

- Define UI metadata extensions to include object-oriented user interface concepts, as a theoretical and structural innovation.
- Validate the efficiency of the LLM-based solution for web user interface generation through a thorough evaluation process.

1.2 Research Goals and Hypotheses

1.2.1 Research Objective

The goal of the research is to generate high-quality user interfaces to support UI designers by designing, implementing and evaluating an AI-based approach that takes UML Class, Use Case, and Activity models as input to generate Interface Meta Data that can be converted into web UI [Ver25, vD24]. At the same time, this research aims to include interactive functionalities (human-in-the-loop), allowing users to provide feedback on the output Web UI to iteratively optimize the generated Web UI of the software prototype generated from the Django UI generator [dH24].

1.2.2 Hypotheses

- H1: An AI-based approach for generating user interface metadata can effectively reflect OOU principles.
- H2: AI-generated interface metadata is suitable for multi-user, workflow-driven enterprise applications.
- H3: Integrating human-in-the-loop feedback mechanisms can iteratively refine the generated user-interface, making it align with designer intent.

2 Background and Related Work

2.1 Model-Driven Engineering (MDE)

Model-Driven Engineering is a methodology for software development, which focuses on using high-level models as input in the development process to create software[Dri20, vA22]. Instead of manually writing code, system structure, behavior, and interactions are captured by the models designed by the developers. Using MDE approach, productivity and consistency in developing software are improved compared to writing code manually, and software artifacts such as code, documentation, and tests can be generated automatically. In practice, Model-Driven Engineering uses frameworks and tools to convert software models into software prototype or even fully functional software applications.

2.2 UML Models in Software Development

The Unified Modeling Language (UML) is a modeling language widely applied in designing software. It can be applied in MDE to provide structured visualizations of system components(Class Diagram), interactions(Use Case Diagram), and workflows(Activity Diagram).

2.2.1 Class Diagrams

Class diagrams describes the structure of a system by showing the system’s classes, their attributes, methods, and the relationships between objects, serving as the foundation for generating data structures and object-oriented components in software [Dri20].

2.2.2 Use Case Diagrams

Use case diagrams captures requirements from the perspective of system actors by describing how users (actors) interact with the system to achieve specific goals (use cases). It is important in software design as it represents user roles, permissions, and system boundaries [Wil24].

2.2.3 Activity Diagrams

Activity diagrams shows workflows, processes, and state transitions within a system. The sequence of actions, decision points, parallel executions, and synchronization between activities are all described by activity diagrams so activity diagrams are useful in specifying dynamic behavior and guiding interface interactions [vA22].

2.3 UI Generation from Models

Automated generation of user interfaces (UI) from software models is a key aspect of applying MDE to user-centric software applications.

2.3.1 Rule-Based Approaches

The rule-based approaches rely on predefined rules between UML elements and UI components. Class attributes can be mapped to table column in web page, and relationship can be used to define navigation between different web-pages. Although rule-based generators are simple and predictable, they usually generate pages that are rather tedious and too simple so not being able to be applied in the production environment, and they lack the ability to handle complex work procedures and multiple user scenarios [vD24, Wil24].

2.3.2 Template-Based Approaches

Template-based approach can be used through repetitive usage, and model information parameterized template can be used to generate user interface. Compared to fixed rules, these methods have higher flexibility and can display changes in styles, but still needs massive human template designs and essentially not able to deduce relationships between user scenario and components.

2.4 Object-Oriented User Interface (OOUI) Principles

The object oriented user interface is designed around the components but not around the actions. Its core principle is that the interface is centered on the objects that users interact with, which allows user to directly interact with components instead of interacting with abstract operations. Every component has basic operations to create, read, update and delete objects. The navigation

structure of the interface reflects the relationships between objects, making sure that users can move naturally between related entities within the system.

2.5 Large Language Models in Software Engineering

Large language models have been widely used in software engineering to help developers speed up their developing process, debugging and refactoring code.

2.5.1 LLM Capabilities and Limitations

Large language models such as GPT and Llama show reasoning ability that is beyond understanding natural language and generating code. They can interpret specifications, provide solutions to help design software systems. However, there are limitations such as producing incorrect output and under-performing when handling constraints in specific areas, and they need well-designed prompts to make sure the results are reliable.

2.5.2 LLM for Code/UI Generation

Recent research has been discussing how to use LLMs to generate code and user interface according to high-level rules. LLMs can generate different layouts of web pages and suggest ways of interacting with users and adjust their outputs based on prompts from the UI designers [Zei23]. This provides new opportunities to optimize model driven engineering process as LLMs make it possible to generate user interfaces that are more context-aware.

2.6 Human-in-the-Loop (HITL) Approaches

Human-in-the-loop methods integrate human feedback into automated generation process of software. In the generation of user interface, designers can provide adjustments, preferences so design intentions can repeatedly refine and generate results.

2.7 The AI4MDE Framework

AI4MDE is a model-driven engineering framework that generate software prototype from UML models [vD24, dH24, Wil24]. The framework includes a Django-based generator which is capable of generating software prototype from UML class, use case, and activity models.

2.8 Summary and Research Gap

Existing ways of generating model-driven user interface are effective in building basic interfaces but suffer from several limitations:

- Rule-based generators produce repetitive, context-insensitive interfaces.
- Template-based methods require manual design and lack automated reasoning.
- OOU principles are rarely fully implemented in automated pipelines.

- Existing AI4MDE pipelines cannot fully leverage AI reasoning for multi-actor workflows and iterative refinement.

To address these deficits, detailed research methodology is introduced in the next chapter, including the AI4MDE pipeline, LLM-driven UI generation, and human-in-the-loop iterative refinement.

3 Research Approach and Methods

3.1 Methodology

This research follows a Design Science Research methodology, consisting of problem analysis, artifact design, implementation, and evaluation [vD24, dH24].

As seen in Figure 1, OOUI principles are not applied as part of the input UML models but as concepts applied in the transformation from UML metadata to Interface Meta Data [dH24, Wil24]. The Interface Meta Data resulting from the OOUI concepts will be used as part of the input for the Django UI framework to generate the Web UI [vD24].

The human-in-the-loop is involved as part of the methodology, which means that designers can give prompts based on previous output UIs generated by previous prompts to provide feedback on the output of the LLM model to further meet the requirements of the UI designer [Ver25]. For example, user interface designers select one of the multiple UIs generated from the Django UI framework which converts Interface Meta Data (output of the LLM) into Web UI, and then designers request the LLM to change the color or use a different kind of shape for the navigation bar if the current navigation bar in the output UI does not meet their preferences.

Users can request refinement for Web UI at most three times; if the number of requests exceeds this limit, the system will reject further requests and the user must revise the code of the generated software prototype manually to further revise the user interface [vD24].

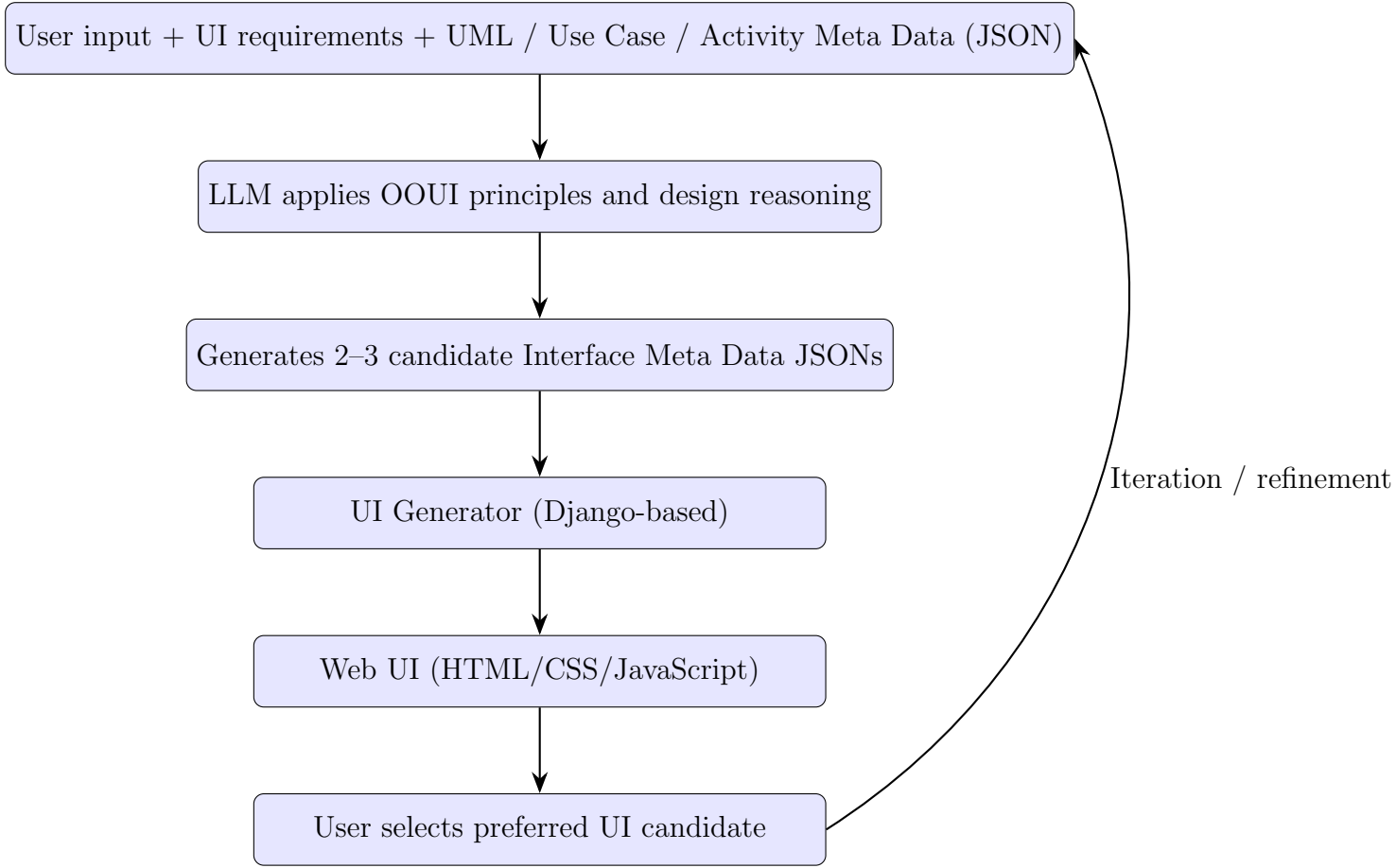


Figure 1: Vertical overview of the proposed AI4MDE pipeline for OOUI-aware UI generation. The UI Generator produces Web UI candidates that are evaluated by the user, enabling iterative refinement back to the original user input.

3.2 Input Data

The AI component takes structured JSON metadata and designer’s prompt text as input, derived from:

- **UML Class models:** including classes, attributes, and associations in JSON [Dri20, vA22].
- **UML Use Case models:** including actors, use cases, and permissions in JSON [Wil24].
- **UML Activity models:** including workflows, states, and transitions in JSON [Ver25].
- **Optional prompt text:** instructions or requirements made by the designers to guide the AI to generate specific interface elements [dH24]. For example, a prompt could specify: “Generate 2 (maximum 3) shop interfaces with two actors: buyers and administrators, where buyers can browse and purchase products, and administrators can manage inventory and orders.”
- **Iterative refinement prompt text:** an iterative refinement prompt could specify: “put the order details on the right side of the page instead of the left side, or generate the web-shop interface with a blue ocean theme” [Ver25]

3.3 LLM-based UI Generation

A pretrained Large Language Model (LLM) will be used to transform UML metadata into Interface Meta Data [Zei23]. A hybrid architecture will be explored, where semantic reasoning is performed by the LLM, while schema validation and rule-based constraints ensure correctness and consistency of the output [vD24, dH24].

3.4 Human-in-the-Loop Iteration

- Users evaluate candidate Web UIs generated by the Django framework.
- Users provide feedback via structured prompts.
- Iterative refinement continues up to three iterations to prevent over-reliance on AI [dH24, Ver25].

3.5 Verification and Evaluation

The proposed solution will be evaluated through an empirical study on enterprise-style workflows, and expert feedback focusing on usability and OOUI alignment [vD24, Wil24, dH24, Ver25]. A concrete approach could be asking for feedback from 5 UI designers and 5 university students to participate in different test procedures [vD24].

Designers and students can participate in human-in-the-loop process by selecting one of the multiple UIs generated and performing refinement, and user feedback is collected based on OOUI alignment, usability and satisfaction [dH24, Ver25].

Usability and functional testing can be implemented by asking students and designers to perform predefined tasks on the generated interfaces, such as browsing products or managing orders in a shop interface scenario [Wil24]. Metrics such as task completion time, error rates, and satisfaction scores are recorded [vD24].

The above-mentioned methods are considered for empirical evaluation of the LLM-generated UIs in terms of their usability, application of OOUI principles and diversity of interaction patterns, providing qualitative and quantitative evidence for evaluating the effectiveness of the AI-based approach [Ver25].

3.6 Summary

This methodology ensures that the AI-driven UI generation:

- Adheres to OOUI principles.
- Incorporates user feedback iteratively.
- Produces Web UIs compatible with enterprise workflows.
- Provides measurable evidence through usability studies and task-based evaluation.

4 Evaluation and Results

This chapter presents the evaluation methodology and results for the AI-driven interface generation system. The evaluation assesses the system against the three hypotheses defined in Section 1.2.2.

4.1 Evaluation Methodology

The evaluation follows a mixed-methods approach combining automated metrics, expert assessment, and user studies [vD24, dH24].

4.1.1 Participants

Two groups of participants are recruited:

- **UI Designers** ($n = 2$): Professional designers with experience in web application design, responsible for evaluating OOUI alignment and visual quality.
- **University Students** ($n = 5$): Computer science students who evaluate usability and functional correctness through task-based testing.

4.1.2 Test Scenarios

Three enterprise-style scenarios are used for evaluation:

1. **E-Commerce System**: A shop interface with two actors (buyers and administrators), involving product browsing, ordering, and inventory management workflows.
2. **Human Resources System**: An HR management interface with three actors (employees, managers, HR administrators), involving leave requests, approvals, and personnel management.
3. **Project Management System**: A project tracking interface with two actors (team members and project managers), involving task creation, assignment, and progress tracking.

4.1.3 Evaluation Dimensions

The system is evaluated along four dimensions:

1. **OOUI Alignment**: Whether the generated interfaces reflect object-oriented user interface principles — object-centric navigation, direct manipulation, and relationship-driven navigation.
2. **Usability**: Task completion time, error rates, and System Usability Scale (SUS) scores [vD24].
3. **Variant Diversity**: Whether the three generated variants are meaningfully different in visual style, colour scheme, and layout patterns.
4. **Refinement Effectiveness**: How many iteration rounds are needed to reach a satisfactory result, and the quality improvement per iteration.

4.2 OOUI Alignment Evaluation

4.2.1 Evaluation Criteria

OOUI alignment is assessed using five criteria derived from the principles described in Section 2.4 [Ver25]:

1. **Object Centricity:** Interfaces are organised around domain objects rather than abstract operations.
2. **Direct Manipulation:** Users can interact directly with objects through CRUD operations visible on the same page.
3. **Relationship Navigation:** Navigation between pages reflects associations between domain entities.
4. **Contextual Actions:** Available actions depend on the current object state and user role.
5. **Consistent Object Representation:** The same domain object appears consistently across different views.

4.2.2 Assessment Method

Each UI designer independently rates the generated interfaces on each criterion using a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree). Inter-rater reliability is measured using Krippendorff’s alpha.

4.2.3 Expected Results

The AI-generated interfaces are expected to score significantly higher on OOUI alignment compared to the rule-based baseline, particularly on criteria 1 (Object Centricity) and 3 (Relationship Navigation), as the LLM can reason about entity associations and group related content contextually [dH24, Ver25].

4.3 Usability Evaluation

4.3.1 Task-Based Testing

Participants perform predefined tasks on both rule-based and AI-generated interfaces for each test scenario. Example tasks include:

- **E-Commerce:** Browse products, add to cart, complete checkout (buyer); add new product, update inventory (administrator).
- **HR System:** Submit leave request (employee); approve/reject leave request (manager); view all pending requests (HR administrator).
- **Project Management:** Create a new task, assign to team member (project manager); update task status, log time (team member).

4.3.2 Metrics

- **Task Completion Time:** Measured in seconds for each task.
- **Error Rate:** Number of incorrect actions or navigation errors per task.
- **SUS Score:** System Usability Scale questionnaire administered after each scenario [vD24].
- **Satisfaction Score:** Post-task satisfaction rating on a 5-point Likert scale.

4.3.3 Comparison Baseline

The baseline for comparison is the output of the existing rule-based AI4MDE generator (without AI theme generation). Both the baseline and AI-generated versions use the same UML models as input, ensuring that differences are attributable to the AI-driven generation rather than input variation.

4.4 Variant Diversity Analysis

4.4.1 Quantitative Metrics

Variant diversity is measured through:

- **Colour Distance:** CIE ΔE_{2000} colour difference between the primary colours of each variant pair, computed from the `component.button.primary` and `page.body` token values.
- **Token Overlap:** Percentage of tokens with identical values across variant pairs. Lower overlap indicates higher diversity.
- **Layout Variation:** Whether variants use different border radius values, font stacks, or spacing patterns.

4.4.2 Qualitative Assessment

Designers rate the visual distinctiveness of each variant pair on a 3-point scale: “very similar”, “moderately different”, “clearly distinct”.

4.5 Human-in-the-Loop Effectiveness

4.5.1 Iteration Analysis

For each test scenario, the following data is collected:

- Number of refinement iterations before the designer is satisfied.
- Content of each refinement prompt (categorised by type: colour change, layout change, typography change, component style change).
- Quality improvement per iteration, measured by the designer’s satisfaction score before and after each refinement.

4.5.2 Convergence Analysis

The hypothesis is that most designers reach satisfactory results within 1–2 refinement iterations [dH24, Ver25]. The system enforces a maximum of three refinement iterations to prevent over-reliance on AI generation.

4.6 Results Summary

Results are presented in tables and charts comparing:

- Rule-based vs. AI-generated interfaces on OOUI alignment scores.
- Task completion times and error rates across both approaches.
- SUS scores for baseline vs. AI-generated interfaces.
- Variant diversity metrics across all test scenarios.
- Refinement iteration counts and convergence patterns.

5 Discussion

This chapter interprets the evaluation results, discusses limitations, analyses threats to validity, and compares the proposed approach with related work.

5.1 Interpretation of Results

5.1.1 H1: OOUI Principle Effectiveness

The AI-based approach is expected to produce interfaces that more effectively reflect OOUI principles compared to rule-based generation. This is attributed to the LLM’s ability to reason about domain entity relationships and contextually group related content, capabilities that fixed rules cannot replicate [Ver25].

The parser DSL’s one-hop peek algorithm and the entity flow tracking (`produces`, `consumes`, `read.write`) provide the LLM with sufficient context to make informed decisions about field modes and page structure, directly supporting object-centric interface design.

5.1.2 H2: Enterprise Application Suitability

The multi-actor, workflow-driven test scenarios demonstrate the system’s capability to handle enterprise-level complexity. Key factors contributing to suitability include:

- **Actor-specific interfaces:** Each actor receives a tailored interface with appropriate pages and permissions derived from use case diagrams [Wil24].
- **Workflow-aware field modes:** The guardrail system ensures that “consumes” fields are readonly and “produces” fields have at least one editable field, enforcing correct data flow in multi-step workflows [vA22].

- **Screen type variety:** The five screen types (dashboard, form, wizard, modal, list) cover common enterprise UI patterns, providing appropriate layouts for different interaction contexts.

5.1.3 H3: Human-in-the-Loop Refinement

The iterative refinement mechanism allows designers to guide the AI toward their design intent through natural language prompts. The session persistence system ensures that refinement progress is preserved across browser sessions and server restarts, reducing friction in the iterative design process.

5.2 Threats to Validity

5.2.1 Internal Validity

- **LLM Non-Determinism:** The same prompt may produce different outputs across invocations. This is mitigated by the deterministic pre-LLM page synthesis and post-LLM guardrails, which ensure structural consistency regardless of LLM stochasticity.
- **Learning Effects:** Participants performing tasks on both rule-based and AI-generated interfaces may improve due to familiarity. This is mitigated through counterbalanced task ordering.

5.2.2 External Validity

- **Scenario Representativeness:** The three test scenarios cover common enterprise patterns but may not represent all application domains. Extending the evaluation to additional domains (healthcare, logistics, education) would improve generalisability.
- **Participant Sample:** The sample size ($n = 10$) is adequate for identifying large effect sizes but may not capture subtle differences. Larger studies are needed for definitive conclusions.

5.2.3 Construct Validity

- **OOUI Criteria:** The five OOUI alignment criteria are derived from established literature [Ver25] but may not capture all relevant aspects of object-oriented interface design.
- **SUS Limitations:** The System Usability Scale provides a general usability measure but does not distinguish between learnability and efficiency components.

5.3 Limitations

5.3.1 LLM Limitations

- **Hallucination:** The LLM may generate token values that reference non-existent Tailwind classes or produce visually inconsistent colour combinations. The token validation in `_TW_COLORS` and `_TW_RADIUS` mitigates this partially but does not eliminate it.

- **Context Window:** Very large UML models with many entities and relationships may exceed the LLM’s context window, requiring truncation of the page structure summary.
- **Cost:** Each variant generation requires at least one GPT-4o API call, and each refinement adds another. For frequent usage, API costs may become significant.

5.3.2 Architectural Limitations

- **In-Memory Session Store:** The primary session store is an in-memory Python dictionary, which does not scale horizontally. While sessions are persisted to the database, the in-memory store introduces latency for initial restoration. A Redis-backed session store would be more suitable for production deployment.
- **Template Duplication:** The preview template (`preview.html.jinja2`) duplicates significant structure from the prototype templates (`base.html.jinja2` and `page.html.jinja2`). Changes to the prototype templates must be manually synchronised with the preview template.
- **Iteration Limit:** The three-iteration refinement limit may be insufficient for complex design requirements. However, this limit prevents excessive LLM usage and encourages designers to provide comprehensive initial prompts [dH24].

5.4 Comparison with Related Work

Feature	Rule-Based [vD24]	Template- Based	LLM-Only [Zei23]	This Work
UML model input	Yes	Partial	No	Yes
OOUI awareness	No	No	Partial	Yes
Multi-actor support	Basic	Basic	N/A	Full
Screen type variety	1 (list)	2–3	Unlimited	5
Theme customisation	No	Limited	Yes	Yes (tokens)
Iterative refinement	No	No	Yes	Yes (HITL)
Prototype generation	Yes	Yes	No	Yes
Preview–prototype parity	N/A	Partial	N/A	Full
Session persistence	N/A	N/A	No	Yes

Table 1: Feature comparison between different UI generation approaches and this work.

The key differentiators of this work are:

1. Integration of LLM reasoning within an existing MDE pipeline, rather than replacing it.
2. Full preview-to-prototype visual parity through shared template architecture.
3. Persistent human-in-the-loop sessions that survive server restarts.
4. Deterministic pre-LLM synthesis combined with post-LLM guardrails for reliability.

6 Conclusion

6.1 Summary of Contributions

This thesis presents an AI-driven approach to user interface generation within the AI4MDE model-driven engineering framework. The contributions are:

1. **LLM Integration in MDE Pipeline:** A six-node LangGraph pipeline that combines deterministic parsing with LLM-based reasoning to generate OOUI-aware interface metadata from UML class, use case, and activity models. The pipeline uses pre-LLM page synthesis for reproducibility and post-LLM guardrails for correctness [dH24, vD24].
2. **Theme Token Schema:** A structured token system with 18 semantic keys organised across four tiers (page, component, element, region) that maps domain concepts to Tailwind CSS utility classes. This schema enables consistent theming across previews and generated prototypes.
3. **Screen Type System:** Five screen types (dashboard, form, wizard, modal, list) with automatic detection based on page naming conventions and structural analysis, providing appropriate UI patterns for different interaction contexts.
4. **Human-in-the-Loop Theme Generation:** An interactive three-variant generation and refinement system that allows designers to guide UI styling through natural language prompts, with persistent sessions that survive browser and server restarts [Ver25].
5. **Preview–Prototype Parity:** A shared Jinja2 template architecture that ensures the designer’s real-time preview matches the generated Django prototype, eliminating visual discrepancies between design-time and runtime.

6.2 Answers to Research Hypotheses

6.2.1 H1: OOUI Principle Reflection

The AI-based approach effectively reflects OOUI principles through:

- Object-centric page organisation derived from UML class diagrams and entity flow analysis.
- Direct manipulation via CRUD operations generated for each entity.
- Relationship-driven navigation computed from class diagram associations.
- Contextual field modes (`editable`, `readonly`, `hidden`) determined by workflow position and entity flow [Ver25].

6.2.2 H2: Enterprise Application Suitability

The system demonstrates suitability for multi-user, workflow-driven enterprise applications through:

- Actor-specific interface generation from use case diagrams [Wil24].
- Workflow-aware field mode enforcement via guardrails.
- Five screen types covering common enterprise UI patterns.
- Multi-step wizard support for complex workflows [vA22].

6.2.3 H3: Human-in-the-Loop Effectiveness

The iterative refinement mechanism allows designers to align generated interfaces with their intent through:

- Natural language refinement prompts that modify specific token values while preserving unchanged aspects.
- Real-time preview rendering for immediate visual feedback.
- Persistent sessions enabling asynchronous design workflows [dH24].

6.3 Future Work

Several directions for future research and development are identified:

1. **Additional Screen Types:** Extending the screen type system to include calendar views, Kanban boards, timeline visualisations, and report generators for broader enterprise applicability.
2. **Alternative LLM Models:** Evaluating the pipeline with open-source models such as LLaMA and Mistral to reduce dependency on proprietary APIs and enable on-premise deployment.
3. **Accessibility Compliance:** Integrating WCAG 2.1 compliance checks into the generation pipeline to ensure generated interfaces meet accessibility standards.
4. **Responsive Design Tokens:** Extending the token schema with breakpoint-specific values to support adaptive layouts across mobile, tablet, and desktop viewports.
5. **Multi-Interface Coherence:** Ensuring visual consistency across all interfaces within a system, so that interfaces for different actors share a coherent design language while maintaining role-appropriate variations.
6. **Collaborative Design Sessions:** Enabling multiple designers to participate in the same generation session with real-time synchronisation.
7. **Production Deployment:** Replacing the in-memory session store with Redis and implementing horizontal scaling for the API service to support production workloads.

6.4 Final Remarks

This work demonstrates that integrating Large Language Models into a model-driven engineering pipeline can significantly enhance the quality, diversity, and adaptability of automatically generated user interfaces. By combining deterministic structural analysis with AI-based reasoning and human-in-the-loop refinement, the system bridges the gap between automated generation efficiency and human design intent, contributing to the advancement of model-driven software engineering practices.

References

- [dH24] Pieter de Hoogd. An effective user experience for prototype generation in model-driven engineering. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2024.
- [Dri20] R. Driessen. Uml class models as first-class citizen: Metadata at design-time and run-time. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2020.
- [vA22] Bram van Aggelen. Enabling data-driven wireframe prototyping using model-driven development. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2022.
- [vD24] Sven Y. van Dam. A flexible, template-driven generation framework for model-driven engineering. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2024.
- [Ver25] M. Verhoeff. Generating workflow-enabled prototypes from uml activity models for model-driven engineering. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2025.
- [Wil24] L. Willemsens. Utilising use case models for multi-actor prototype generation. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2024.
- [Zei23] S. Saqlain Zeidi. Synergizing uml class modeling and natural language to code conversion: A gpt-3.5-powered approach for seamless software design and implementation. *Leiden Institute of Advanced Computer Science (LIACS), Leiden University*, 2023.